

Raspberry Pi

Service Reliability Guide

DOOR-LIGHT-LOCK-SERVICE

Setup, Logging & Reliability Hardening

What this guide covers

This guide walks you through every step needed to make your Raspberry Pi RFID door/light/lock system run reliably as a production service. It covers rotating log setup, systemd service hardening, MQTT reconnection, SD card protection, and ongoing maintenance.

Priority Summary

Priority	Action	Effort	Impact
 HIGH	Add logger.py and update all Python scripts	30 min	Prevents log buildup
 HIGH	Limit systemd journal size	5 min	Protects SD card
 HIGH	Harden all 4 systemd service files	15 min	Auto-recovery on crash
 HIGH	Add MQTT on_disconnect handler	10 min	Survives broker restart
 MEDIUM	Install sdnotify watchdog	15 min	Detects frozen process
 MEDIUM	Move logs to tmpfs RAM disk	10 min	Reduces SD writes
 LOW	Add healthcheck cron job	5 min	Extra safety net
 LOW	Set noatime on filesystem	5 min	Reduces SD wear

STEP 1 — Rotating Log Setup

1.1 Create the logger.py Library File

This is a new shared helper that all your Python services will import. It creates rotating log files that automatically delete old data to protect your SD card.

Deploy location:

```
/home/alice/broker/library/logger.py
```

Create the logs directory:

```
mkdir -p /home/alice/broker/logs
```

 *logger.py is provided as a separate file in the revised code package. Copy it to the library/ folder.*

Log rotation settings (inside logger.py)

Setting	Value
Max file size	2 MB per log file
Backup copies	3 old files kept
Total per service	Max 8 MB
Total all services	Max ~50 MB
Log directory	/home/alice/broker/logs/

1.2 Update All Python Scripts

Replace all print() calls with the logger. All revised scripts are provided in the code package. Deploy them as follows:

Main service scripts (deploy to /home/alice/broker/)

Script	Logger name
run.py	get_logger("run")
switchStatus.py	get_logger("switch")
radarScanControl.py	get_logger("radar")
lightLockControl.py	get_logger("lightlock")

Library scripts (deploy to /home/alice/broker/library/)

Script	Logger name
mqttts.py	get_logger("mqttts")
startstops.py	get_logger("startstops")
status.py	get_logger("status")

utilitys.py	get_logger("utilitys")
reader.py	get_logger("reader")

Log levels used

Level	Used for
log.info	Normal events — door open/close, MQTT received, scan start/stop
log.debug	High-frequency messages — GPIO polling, raw serial data
log.warning	Unexpected but non-fatal — mutex busy, mqttc is None
log.error	Failures — JSON parse error, serial error, publish failed

1.3 Limit Systemd Journal Size

This is the most critical SD card protection step. Without this, journald will fill your SD card over time.

Edit the journald configuration:

```
sudo nano /etc/systemd/journal.conf
```

Add or uncomment these lines:

```
[Journal]
SystemMaxUse=100M
SystemMaxFileSize=10M
MaxRetentionSec=7day
Compress=yes
```

Apply the changes:

```
sudo systemctl restart systemd-journal
```

⚠ Without setting SystemMaxUse, journald can grow to fill the entire SD card. This is the #1 cause of Raspberry Pi SD corruption in always-on systems.

1.4 Move Logs to RAM Disk (tmpfs)

Writing logs to RAM instead of the SD card reduces SD card wear and extends its lifespan significantly.

Add this line to /etc/fstab:

```
sudo nano /etc/fstab
tmpfs /home/alice/broker/logs tmpfs defaults,noatime,nosuid,size=50m 0 0
```

Apply without rebooting:

```
sudo mount -a
```

⚠ Logs in tmpfs are lost on reboot. This is acceptable for debug logs. If you need logs to survive reboots, skip this step and keep logs on the SD card but rely on the rotation settings in logger.py to keep sizes small.

STEP 2 — Systemd Service Hardening

2.1 Harden All Four Service Files

Update each service file with the improved settings below. The key additions are: a proper restart policy, startup ordering protection, and a watchdog timeout.

send.service (run.py — main entry point)

```
sudo nano /etc/systemd/system/send.service

[Unit]
Description=Send Flask App
After=network-online.target mosquitto.service
Wants=network-online.target mosquitto.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=alice
WorkingDirectory=/home/alice/broker
ExecStart=/bin/bash -c "sleep 10 && /home/alice/broker/venv/bin/python
/home/alice/broker/run.py"
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=10
WatchdogSec=120
KillMode=mixed
TimeoutStopSec=15
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

switch.service (switchStatus.py — door monitor)

```
sudo nano /etc/systemd/system/switch.service

[Unit]
Description=Door Switch Monitor
After=network-online.target mosquitto.service send.service
Wants=network-online.target mosquitto.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=alice
WorkingDirectory=/home/alice/broker
ExecStart=/bin/bash -c "sleep 20 && /home/alice/broker/venv/bin/python
/home/alice/broker/switchStatus.py"
```

```
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=10
WatchdogSec=120
KillMode=mixed
TimeoutStopSec=15
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

radarscan.service (radarScanControl.py)

```
sudo nano /etc/systemd/system/radarscan.service
```

```
[Unit]
Description=RFID Radar Scan Control
After=network-online.target mosquitto.service send.service
Wants=network-online.target mosquitto.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=alice
WorkingDirectory=/home/alice/broker
ExecStart=/bin/bash -c "sleep 25 && /home/alice/broker/venv/bin/python
/home/alice/broker/radarScanControl.py"
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=10
WatchdogSec=120
KillMode=mixed
TimeoutStopSec=15
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

lightlock.service (lightLockControl.py)

```
sudo nano /etc/systemd/system/lightlock.service
```

```
[Unit]
Description=Light and Lock MQTT Control
After=network-online.target mosquitto.service send.service
Wants=network-online.target mosquitto.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=alice
```

```
WorkingDirectory=/home/alice/broker
ExecStart=/bin/bash -c "sleep 12 && /home/alice/broker/venv/bin/python
/home/alice/broker/lightLockControl.py"
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=10
WatchdogSec=120
KillMode=mixed
TimeoutStopSec=15
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

2.2 What the New Settings Do

Setting	Purpose
StartLimitIntervalSec=60	If service crashes 5 times in 60s, systemd stops retrying to prevent reboot loop
StartLimitBurst=5	Number of allowed restarts within the interval above
RestartSec=10	Wait 10 seconds between restart attempts (was 5s)
WatchdogSec=120	Systemd kills and restarts the service if it stops responding for 2 minutes
KillMode=mixed	SIGTERM to main process first, then SIGKILL to child processes
TimeoutStopSec=15	Give the process 15 seconds to clean up before force kill
After=...send.service	switch/radar/lightlock wait for send.service (run.py) to start first

2.3 Apply the Changes

After editing the service files, reload and restart:

```
sudo systemctl daemon-reload

sudo systemctl restart send.service
sudo systemctl restart radarscan
sudo systemctl restart lightlock
sudo systemctl restart switch
```

Verify all are running:

```
sudo systemctl status send radarscan lightlock switch
```

STEP 3 — MQTT Reconnection

3.1 Why This Is Needed

If the Mosquitto broker restarts (e.g. after a power cut or crash), your Python services will lose their MQTT connection and never recover — they will stop receiving scan commands and stop publishing status. The fix is to add an `on_disconnect` handler and configure reconnect backoff.

3.2 Add to `mqttts.py`

In the `mqtt_setup_safe()` function, add the following two lines and the `on_disconnect` callback:

```
def on_disconnect(client, userdata, rc):
    if rc != 0:
        log.warning(f"[MQTT] Unexpected disconnect rc={rc}, will auto-reconnect...")
    else:
        log.info("[MQTT] Disconnected cleanly")

def mqtt_setup_safe(client_suffix="main"):
    global mqttc
    client_id = f"{HOSTNAME}-{client_suffix}"
    mqttc = mqttts.Client(client_id=client_id, clean_session=True)
    mqttc.reconnect_delay_set(min_delay=1, max_delay=30) # ADD THIS
    mqttc.on_disconnect = on_disconnect                # ADD THIS
    mqttc.on_connect = on_connect
    mqttc.on_message = on_message
    mqttc.connect(BROKER, 1883, 60)
    mqttc.loop_start()
```

 `loop_start()` already handles automatic reconnection internally. The `reconnect_delay_set()` call just controls the timing — it will retry after 1 second, then 2, 4, 8... up to a maximum of 30 seconds.

3.3 Add to `lightLockControl.py`, `switchStatus.py`, `radarScanControl.py`

Add the same `on_disconnect` handler to each script that uses its own MQTT client:

```
def on_disconnect(client, userdata, rc):
    if rc != 0:
        log.warning(f"[MQTT] Unexpected disconnect rc={rc}, reconnecting...")
```

And in the setup section:

```
mqttc.reconnect_delay_set(min_delay=1, max_delay=30)
mqttc.on_disconnect = on_disconnect
```

3.4 Make Mosquitto Reliable Too

Ensure Mosquitto itself restarts on failure:

```
sudo nano /etc/systemd/system/mosquitto.service.d/override.conf
```

```
[Service]
Restart=always
RestartSec=5
```

```
sudo systemctl daemon-reload
sudo systemctl enable mosquitto
sudo systemctl restart mosquitto
```

STEP 4 — Watchdog (Optional but Recommended)

4.1 What Is a Watchdog?

The `systemd WatchdogSec` setting in the service files tells `systemd` to kill and restart a service if it stops sending heartbeat pings. Without the watchdog ping from inside your Python script, setting `WatchdogSec` alone has no effect — the process must actively send pings.

4.2 Install `snotify`

```
source /home/alice/broker/venv/bin/activate
pip install snotify
```

4.3 Add Watchdog Thread to `run.py`

Add this function and thread to `run.py`:

```
import snotify

def watchdog_worker():
    n = snotify.SystemdNotifier()
    n.notify("READY=1")
    log.info("[WATCHDOG] Started, sending heartbeat every 30s")
    while True:
        n.notify("WATCHDOG=1")
        log.debug("[WATCHDOG] Ping sent")
        time.sleep(30)  # must be less than WatchdogSec / 2
```

In the `main()` section, start the thread:

```
threading.Thread(target=watchdog_worker, daemon=True).start()
```

 *The watchdog ping interval (`time.sleep` value) must be less than half of `WatchdogSec`. Since `WatchdogSec=120`, the sleep must be less than 60 seconds. 30 seconds is a safe value.*

STEP 5 — SD Card Protection

5.1 Enable `noatime` on the Filesystem

By default, Linux updates the access timestamp on every file read. On a Raspberry Pi, this causes constant SD card writes even when you are only reading files. Disabling it reduces SD card wear.

```
sudo nano /etc/fstab
```

Find the root filesystem line (it looks similar to this):

```
PARTUUID=xxxxxxxx-02 / ext4 defaults,noatime 0 1
```

Change `'defaults'` to `'defaults,noatime'` if not already set. Then apply:

```
sudo mount -o remount,noatime /
```

5.2 Enable Journal Checksums

This makes the ext4 journal more resilient to corruption from sudden power loss (which is common on Raspberry Pi when power is removed):

```
sudo tune2fs -o journal_data /dev/mmcblk0p2
```

 Replace /dev/mmcblk0p2 with your actual root partition. Run 'df -h /' to confirm the device name.

5.3 Use a Quality SD Card

Hardware quality matters more than any software setting. Use a card rated for endurance writes:

Category	Recommendation
Recommended	SanDisk Endurance, Samsung PRO Endurance — rated for 40,000+ hours
Avoid	Generic no-brand cards, old cards, cheap phone cards
Minimum class	Class 10 / UHS-I Speed Class 3 (U3) or better
Capacity	16GB minimum — gives more spare area for wear levelling

STEP 6 — Health Check & Monitoring

6.1 Create Healthcheck Script

This script runs every 5 minutes via cron and restarts any service that has stopped:

```
sudo nano /home/alice/broker/healthcheck.sh

#!/bin/bash
SERVICES="send switch radarscan lightlock"
for svc in $SERVICES; do
    if ! systemctl is-active --quiet $svc; then
        echo "[$(date)] $svc is DOWN - restarting"
        systemctl restart $svc
    fi
done

chmod +x /home/alice/broker/healthcheck.sh
```

6.2 Add to Crontab

```
sudo crontab -e
```

Add this line at the bottom:

```
* /5 * * * * /bin/bash /home/alice/broker/healthcheck.sh >>
/home/alice/broker/logs/healthcheck.log 2>&1
```

6.3 Useful Monitoring Commands

Task	Command
View live logs for a service	sudo journalctl -fu send.service
View last 50 lines of logs	sudo journalctl -xeu radarscan --no-pager tail -50
Check all service statuses	sudo systemctl status send radarscan lightlock switch
View rotating log file	tail -f /home/alice/broker/logs/run.log
View all log files	ls -lh /home/alice/broker/logs/
Check journal disk usage	journalctl --disk-usage
Check SD card space	df -h /
View healthcheck log	tail -30 /home/alice/broker/logs/healthcheck.log

STEP 7 — Final Deployment Checklist

7.1 Deploy Files

1. Copy logger.py to /home/alice/broker/library/

- 2. Copy revised library scripts to /home/alice/broker/library/
 mqttts.py startstops.py status.py utilitys.py reader.py
- 3. Copy revised main scripts to /home/alice/broker/
 run.py switchStatus.py radarScanControl.py lightLockControl.py
- 4. Create logs directory
 mkdir -p /home/alice/broker/logs
- 5. Create healthcheck script and make executable
- 6. **Update all 4 systemd service files**
- 7. **Edit /etc/systemd/journald.conf to limit journal size**
- 8. Edit /etc/fstab for noatime
- 9. Install sdnotify in virtualenv (optional)

7.2 Apply All Changes

```
sudo systemctl daemon-reload
sudo systemctl restart mosquitto
sudo systemctl restart send radarscan lightlock switch

sudo systemctl enable send radarscan lightlock switch mosquitto
```

7.3 Verify Everything Is Running

```
sudo systemctl status send radarscan lightlock switch

tail -f /home/alice/broker/logs/run.log
```

✔ All 4 services should show 'active (running)'. If any show 'failed', check the logs with: `sudo journalctl -xeu <service-name> --no-pager | tail -30`

Reference — Service Startup Order

Service	Role
1. mosquitto	MQTT broker — must start first, all others depend on it
2. send (sleep 10)	run.py — main service, sets up MQTT, GPIO init, starts workers
3. lightlock (sleep 12)	lightLockControl.py — subscribes to light/lock commands
4. radarscan (sleep 25)	radarScanControl.py — waits for door events to start scan
5. switch (sleep 20)	switchStatus.py — monitors door GPIO, publishes open/close events

Reference — Troubleshooting Quick Guide

Symptom	Fix
Door open/close not triggering scan	Check switch.service logs — look for 'mqttc is None'. Increase sleep delay in switch.service to 25s.

Service not starting	Run: <code>sudo journalctl -xeu <service> --no-pager tail -30</code>
MQTT not reconnecting after broker restart	Ensure <code>on_disconnect</code> handler is added and <code>loop_start()</code> is used (not <code>loop_forever()</code>)
SD card filling up	Check: <code>journalctl --disk-usage</code> and <code>df -h /</code> . Set <code>SystemMaxUse=100M</code> in <code>journald.conf</code>
Service stuck / not responding	Check <code>WatchdogSec</code> is set and <code>sdnotify watchdog</code> thread is running in <code>run.py</code>
Logs not appearing in <code>/logs/</code>	Ensure <code>mkdir -p /home/alice/broker/logs</code> was run and <code>logger.py</code> is in library/